

Virtualizzazione

Antonio Pecchia

antonio.pecchia@unisannio.it

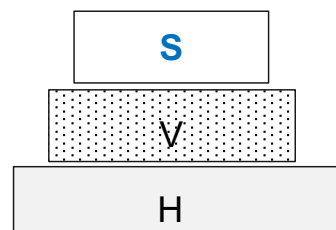
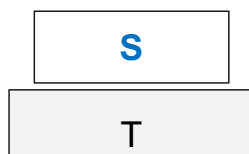
Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

Virtualizzazione

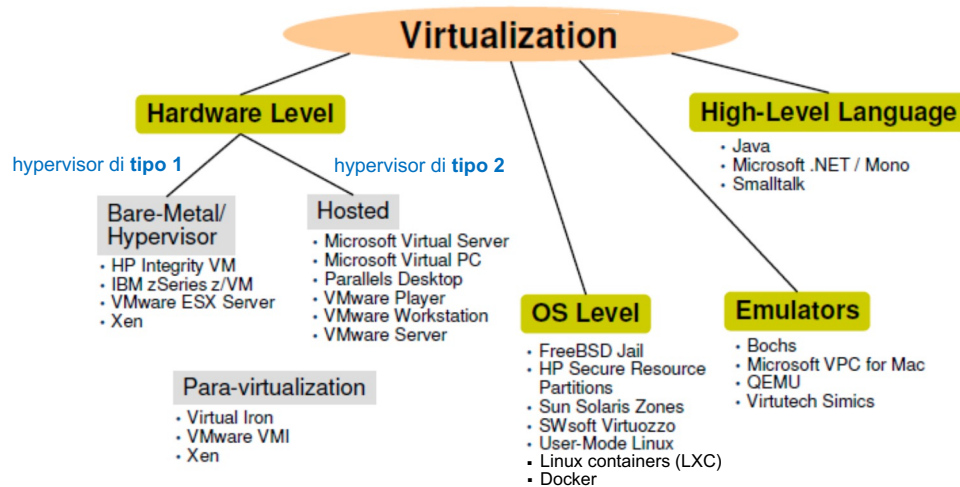
- Uno scenario tipico (non l'unico):
 - sia **S** un software che gira su un hardware **T** (target);
 - sia **V** (**virtual**) un software che gira su hardware **H** (host);
- Si vuole sostituire T (vero) con V (virtuale), ed assicurare che S funzioni "come prima".



Virtualizzazione: alcune motivazioni

- ❑ **Non** si vuole modificare S e sono presenti una o più delle seguenti condizioni:
 - l'hardware T non è disponibile;
 - V è meno costoso di T;
 - V è più flessibile di T;
 - V offre un buon modello di protezione per S.
- ❑ In alcuni casi, S viene modificato: in tal caso, si parla di *paravirtualizzazione*.

Una possibile tassonomia



* spesso con *virtualizzazione* si intende solo quella hardware-level

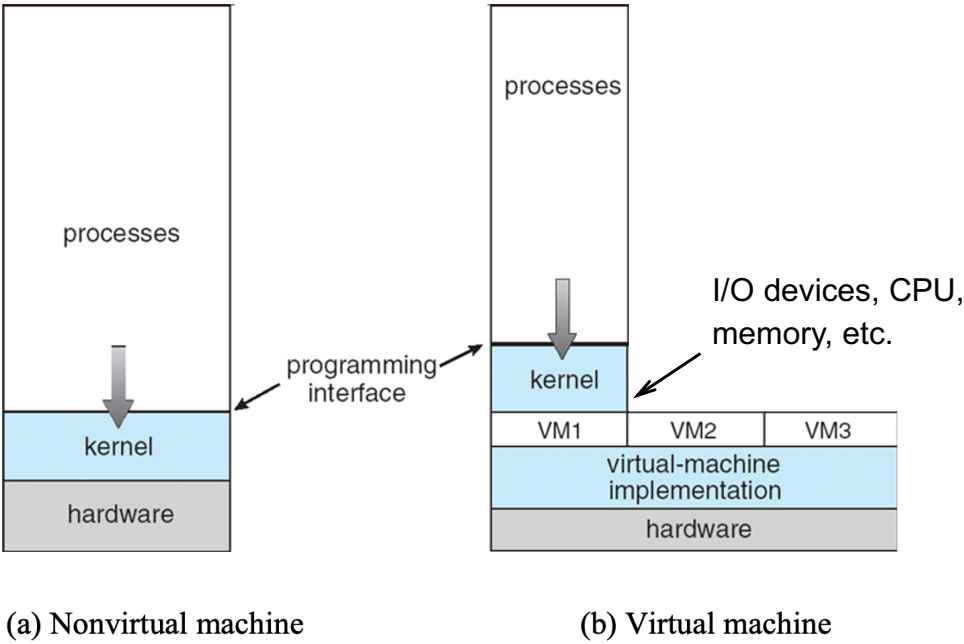
VIRTUAL MACHINE MONITOR

Cap 7 par. da 7.1 a 7.5 (escluso 7.5.2)

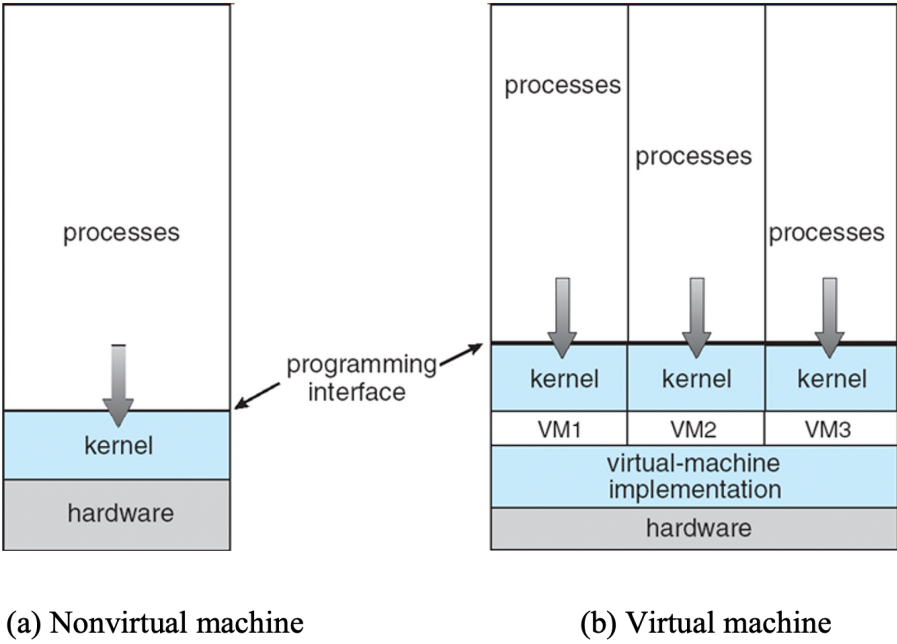
Virtual Machine Monitor

- Un **virtual machine monitor** (VMM, detto anche **hypervisor**) crea l'*illusione* di più **macchine** (**virtuali**), ossia **virtual machine** (VM), in esecuzione sul medesimo hardware fisico;
- I "partecipanti":
 - **host**: la piattaforma hardware sottostante;
 - **VMM**: crea ed esegue macchine virtuali fornendo una interfaccia identica a quella dell'host;
 - **guest**: processo a cui è fornita una 'copia' virtuale dell'host:
 - in generale, un sistema operativo.

Virtual Machine



Virtual Machine



Requisiti di un hypervisor

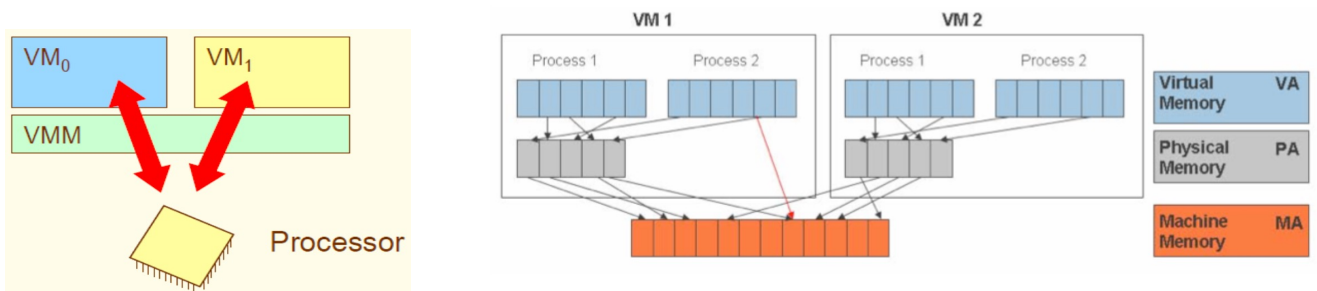
1. **Sicurezza.** L'hypervisor deve avere il controllo completo delle risorse virtualizzate.
2. **Equivalenza.** Il comportamento di un programma in esecuzione all'interno di una macchina virtuale dev'essere identico a quello che lo stesso programma avrebbe in esecuzione sul nudo hardware.
3. **Efficienza.** La maggior parte del codice in esecuzione sulla macchina virtuale dovrebbe funzionare senza interventi da parte dell'hypervisor.

Caratteristiche di un VMM

- Un **VMM**, presenta una interfaccia (virtuale) della piattaforma ai guest;
- Un VMM deve essere **protetto** dal software guest, ed **isolare** gli stack guest (SO+applicazioni);
- Un VMM controlla gli accessi a CPU, memoria e dispositivi di I/O. **Modalità** con cui un VMM crea l'*illusione* di possesso di risorse ai guest:
 - multiplexing temporale;
 - partizionamento delle risorse;
 - mediazione.

Caratteristiche di un VMM (esempi)

- Alla VM è consentito accesso diretto alla risorsa per un certo periodo di tempo (per es., nel caso della CPU);
- Il VMM è responsabile di mappare la memoria fisica del guest sulla memoria vera e propria della macchina.



Hypervisor di tipo 1 e tipo 2

- In generale, è possibile distinguere due approcci:
 - hypervisor di **tipo 1** (anche detto *nativo* o *bare-metal*);
 - hypervisor di **tipo 2** (anche detto *hosted*).

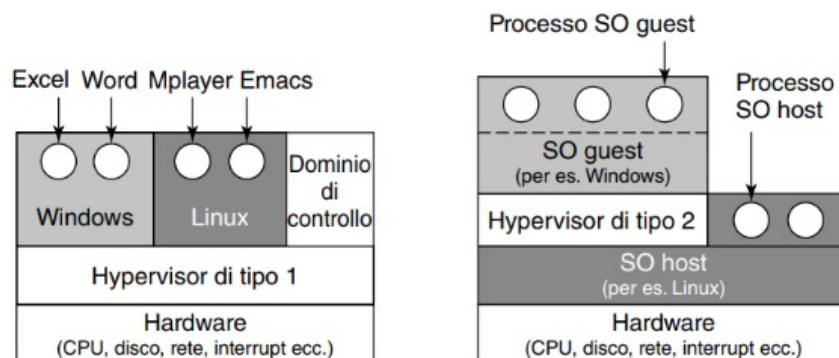


Figura 7.1 Posizione degli hypervisor di tipo 1 e di tipo 2.

Hypervisor di tipo 1 e tipo 2

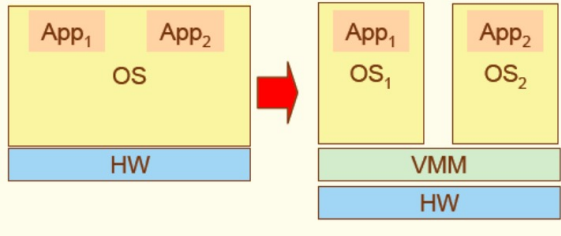
- Un **hypervisor di tipo1** è 'simile' ad un SO poiché è il solo programma in esecuzione nella modalità più privilegiata:
 - un SO '*special purpose*', le cui 'applicazioni' sono i guest;
 - piuttosto che fornire l'interfaccia delle system call, *crea*, *esegue*, e *gestisce* SO guest.
 - la macchina virtuale viene eseguita come processo in modalità utente.
- Un **hypervisor di tipo2** è "semplicemente" un programma eseguito e gestito da un SO host (per es. Linux o Windows).

Motivazioni della virtualizzazione

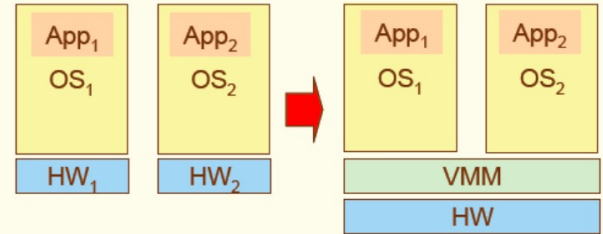
- Le **organizzazioni IT** gestiscono molti servizi applicativi (email, database, web, FTP, ...), ciascuno con le proprie configurazioni:
 - installare e combinare tutto su un singolo server è difficile;
 - aggiornare HW o SO di un sistema è complesso;
 - difficoltà nel bilanciamento del carico;
 - ...
- Le macchine attuali sono fornite di tantissime risorse HW, ed è difficile che un'applicazione server utilizzi tutte le risorse.

Alcune possibilità offerte dalla virtualizzazione

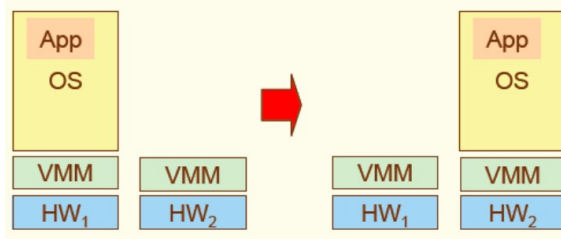
Workload Isolation



Workload Aggregation



Workload Migration



*VMM=virtual machine monitor

Genoa-X Power

1U Rack Server with brand-new AMD EPYC 9004 CPUs up to 128 Cores

NEW



RECT™ RS-8539N12

now with latest AMD EPYC™ 9004 processors - up to 128 cores and 256 Threads:

- **NEW:** Single socket up to AMD EPYC 9754 (128 Cores, 2.25 GHz, 256 MB Cache)
- up to 3 TB DDR5-4800 ECC RAM
- up to 184 TB storage with SSDs (NVMe/SATA/SAS*) in 12 hot-swap trays
- three PCIe 5.0 expansion slots and one OCP 3.0 slot



<https://risingracks.com/blog/beginners-guide-to-server-racks/>



Google data center, Oregon

TECNICHE DI VIRTUALIZZAZIONE

Tecniche di virtualizzazione

- Esistono diverse possibilità*:
 - emulazione;
 - trap-and-emulate (con supporto hardware);
 - binary translation;
 - paravirtualization.

***NOTA:** il focus della presentazione è sulla virtualizzazione del **processore**.

Emulazione

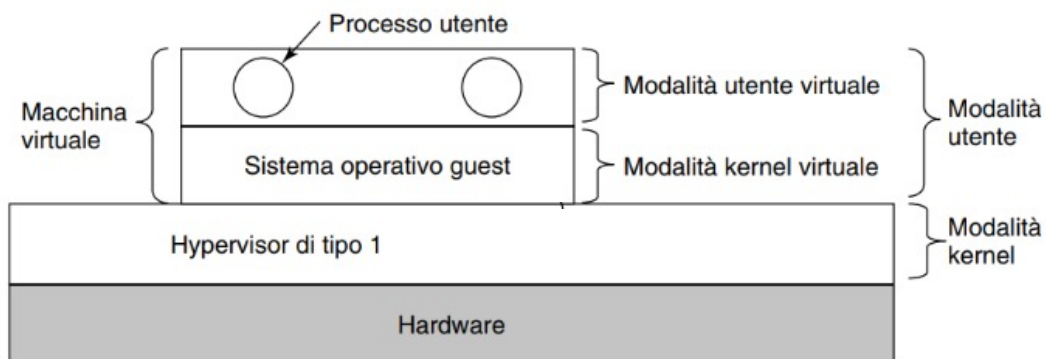
- Realizza ciò che fa il processore, ma **in software**:
 - un emulatore è un programma basato su un loop che *imita* le azioni del processore target: fetch, decode, **execute**;
 - la execute prevede uno **switch** con un **case** per ogni istruzione target. Esempio:

istruzione: `addl %ebx, %eax` `/* eax += ebx */`

emulazione: `enum {EAX=0, EBX=1, ECX=2, ECX=3, ...};`
 `unsigned long regs[8];`
 `regs[EAX] += regs[EBX];`

Sidenote

- Hypervisor di tipo 1



Problema: il SO guest (che "pensa" di trovarsi in modalità kernel) esegue una istruzione consentita solo quando la CPU si trova in modalità kernel.

Sidenote

- Nelle CPU Intel sono previsti 4 livelli di privilegio (ring).

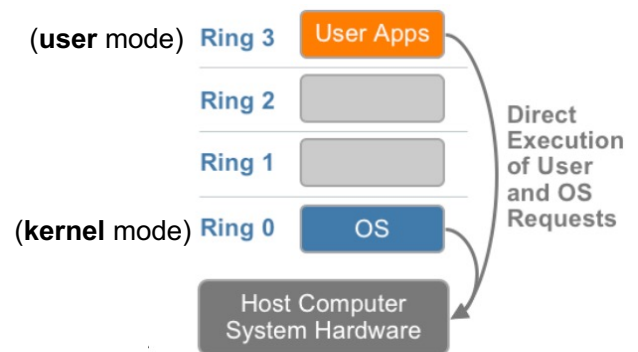
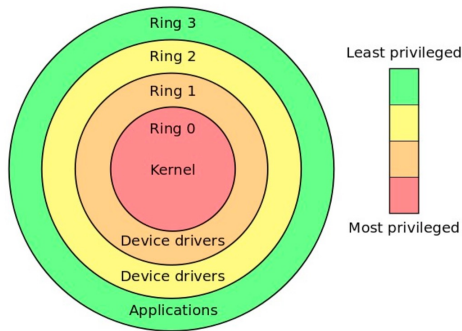
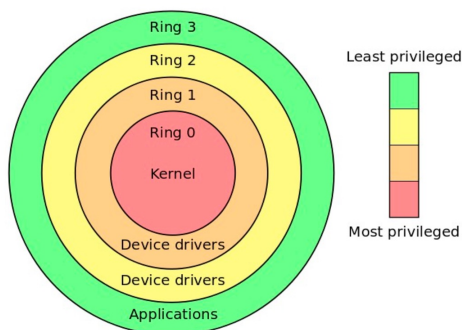


Figure 4 – x86 privilege level architecture without virtualization

Source (Fig.4, ed altre successive): VMware white paper: Understanding Full Virtualization, Paravirtualization, and Hardware Assist (techreport 2008)

Sidenote

- Le invocazioni di funzioni tra i ring possono avvenire solo con meccanismi *hardware-enforced* (per. es. le system call).



- solo il ring 0 può eseguire istruzioni privilegiate (i ring 1/2/3, invece, provocano una trap);
- nel caso tipico, il S.O. esegue in ring 0, le applicazioni in ring 3.

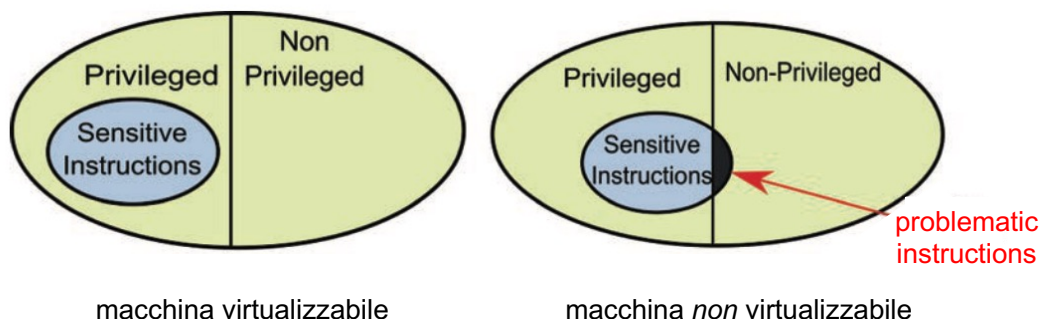
Source (Fig.4, ed altre successive): VMware white paper: Understanding Full Virtualization, Paravirtualization, and Hardware Assist (techreport 2008)

Popek e Goldberg (1974)*

- Ogni CPU con modalità user/kernel ha istruzioni che si comportano in maniera diversa a seconda della modalità in cui sono eseguite (I/O, modifica MMU, etc):
 - tali istruzioni sono definite istruzioni **sensibili**;
- Esistono istruzioni che causano una trap se eseguite in modalità utente (istruzioni **privilegiate**).
- **Una macchina è virtualizzabile solo se le istruzioni sensibili sono un sottoinsieme di quelle privilegiate.**

* Popek, G. J.; Goldberg, R. P. (July 1974). "Formal requirements for virtualizable third generation architectures". Communications of the ACM. 17 (7): 412– 421. doi:10.1145/361011.361073.

Popek e Goldberg (1974)*



Esempio: l'istruzione **popf** sostituisce il registro dei flag, che modifica il *bit* che abilita/disabilita gli interrupt (in modalità utente, tale bit non viene modificato).

Problemi dell'architettura x86

- Diverse istruzioni sensibili **non** generavano trap:
 - ben 17 istruzioni **sensibili**, non privilegiate;

Group	Instructions
Access to interrupt flag	pushf, popf, iret
Visibility into segment descriptors	lar, verr, verw, lsl
Segment manipulation instructions	pop <seg>, push <seg>, mov <seg>
Read-only access to privileged state	sgdt, sldt, sidt, smsw
Interrupt and gate instructions	fcall, longjump, retfar, str, int <n>

- Problema risolto nel **2005**, quando Intel e AMD introdussero la virtualizzazione sulle loro CPU: VT (Intel), SVM (AMD)*.

* Virtualization Technology (VT); Secure Virtual Machine (SVM).

Trap-and-emulate

- Molto del codice eseguito direttamente sulla CPU;
- Il SO esegue in modalità *de-privileged*;
- **Trap** delle istruzioni privilegiate nel VMM;
- Il VMM emula le istruzioni rispetto allo stato virtuale.

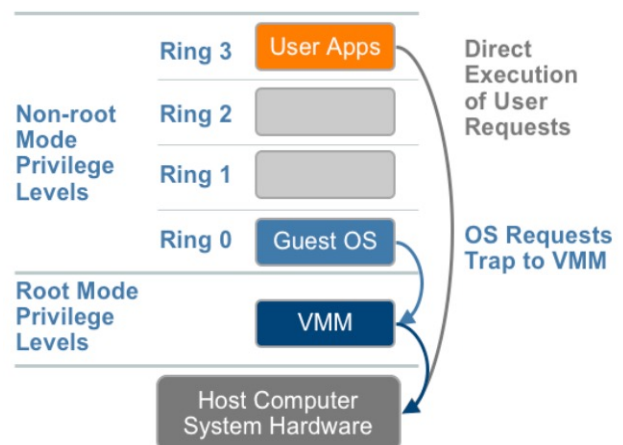


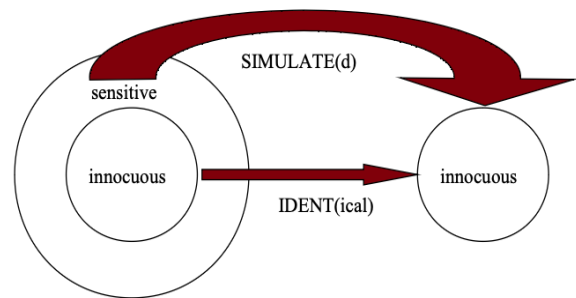
Figure 7 – The hardware assist approach to x86 virtualization

```
cli                                ; disable interrupts
↓
and $0xfd, %gs:vcpu.flags ; disable interrupts
```

traduzione di una `cli` in una modifica di una struttura dati *VMM-level*, parte dell'immagine in memoria della CPU virtuale guest

Binary translation (usata da VMware nel '99)

- ❑ Fino al 2005 x86 non era virtualizzabile, ma VMware ha lanciato il suo primo hypervisor x86 nel 1999 ... (?)
- ❑ Soluzione: **binary translation** delle istruzioni **sensibili** del kernel del guest:
 - tali istruzioni sono sostituite con una chiamata ad una procedura dell'hypervisor che emette una sequenza di **istruzioni controllate**;
 - traduzioni tenute in una **cache**;
 - applicabile anche alle istruzioni *privilegiate* del kernel guest.



Binary translation (usata da VMware nel '99)

- ❑ Utilizzo del **Ring 1**:
 - il guest OS ha più privilegi rispetto ai processi utente;
 - le istruzioni privilegiate nel guest generano una trap;

esecuzione **diretta**, non privilegiata, del codice applicativo **utente** (NOTA: *non* è necessario sostituire le istruzioni sensibili del programma utente).

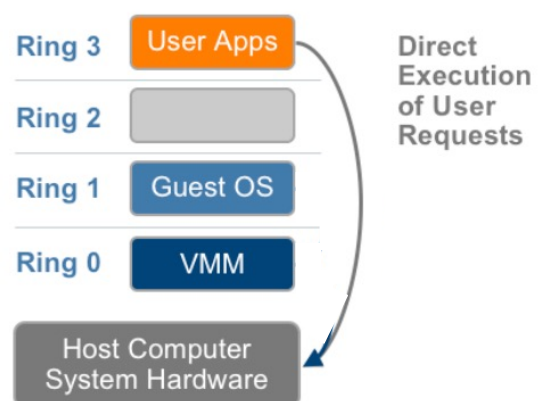


Figure 5 – The binary translation approach to x86 virtualization

Binary translation (usata da VMware nel '99)

- La traduzione binaria avviene quando il guest OS viene invocato.

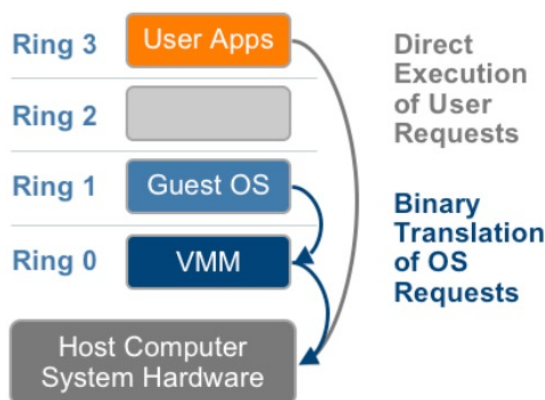
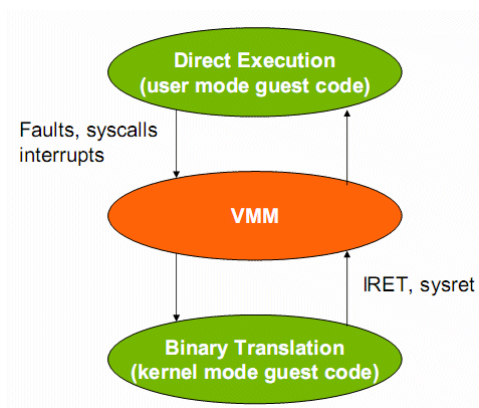
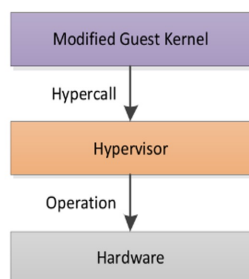


Figure 5 – The binary translation approach to x86 virtualization

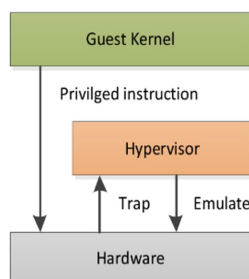
Paravirtualization

- Il guest OS viene **modificato** al fine di **semplificare** alcuni aspetti:
 - per es., i driver parlano direttamente con il VMM, anziché usare trap sulle istruzioni PIO/MMIO;
- Il guest OS è "consapevole" di essere virtualizzato.

Para-virtualization



"Classical" Full-virtualization



Paravirtualization

- Presenza di una interfaccia software (insieme di **hypercall**) che consente al guest di inviare richieste "esplicite" all'hypervisor;
- Il guest usa le hypercall per svolgere operazioni privilegiate sensibili.

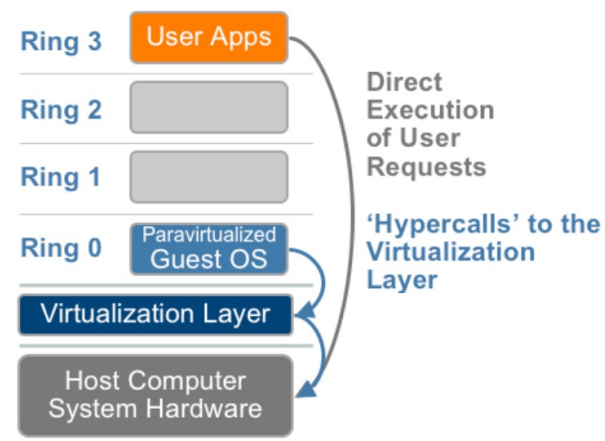


Figure 6 – The Paravirtualization approach to x86 Virtualization

Esempi di hypervisor

Metodo di virtualizzazione	Hypervisor di tipo 1	Hypervisor di tipo 2
Virtualizzazione senza supporto hardware	ESX Server 1.0	VMware Workstation 1
Paravirtualizzazione	Xen 1.0	VirtualBox 5.0+
Virtualizzazione con supporto hardware	vSphere, Xen, Hyper-V	VMware Fusion, KVM, Parallels

Figura 7.2 Esempi di hypervisor. Gli hypervisor di tipo 1 vengono eseguiti direttamente sull'hardware, mentre quelli di tipo 2 utilizzano i servizi offerti da un sistema operativo host installato.